TEM Workbench: code review request

Prepared 3 November 2026 for Claude. Code snapshot when prepared: `cb4ca63`.

Repository: github.com/pKai3/tem-workbench

Please review TEM Workbench, focusing on scientific correctness, automatic peak detection, and performance. Review the current `main` branch and record the commit reviewed. Start with `AGENTS.md` and `docs/implementation-plan.md`.

Please audit and propose improvements first. Do not make broad changes simply to increase scores or produce more passing classifications.

Context

The application analyzes real-space TEM image crops through FFTs, and direct diffraction images through a separate detector. It detects reciprocal-space peaks, compares crystal templates, fits rotation and lattice dimensions, and ranks candidates using positional agreement, scaling penalties, weak-peak evidence and missing-prediction penalties.

It supports HCP, BCC, FCC and generic primitive orthorhombic lattices. Spatial analysis includes adaptive grids and overlapping-window stability maps. Outputs are candidate interpretations, not established phase identities or measured phase fractions.

1. Auto-mode efficiency

Inspect `engine.py`, particularly `analyze()` and `_fit_spectrum()`, alongside `analysis_core.py`, `scale_fitting.py`, `peak_evidence.py` and `parallel_execution.py`.

- **Manual:** analyzes one selected minimum-strength threshold.
- **Auto - rank:** computes one FFT and peak pool, then tests distinct strength-filtered peak sets with crystal matching and evidence scoring.
- **Auto - geometry:** evaluates strength-filtered sets using geometry-only scoring, then runs crystal matching on the selected set.

Please profile representative cases and identify where time actually goes. Consider:

- What can be cached or reused across nested peak sets?
- Are weak-maxima preparation, geometric searches, template transformations or candidate refinements repeated unnecessarily?
- Can candidate screening or incremental updates reduce work without silently losing weaker patterns?
- Are there duplicated geometry calculations after threshold selection?
- Would coarse-to-fine threshold search be reliable, or could it miss important changes?
- Are process overhead, data transfer, memory use or numeric-library threads limiting grid throughput?

Separate optimizations that preserve results from approximations that change the search. Provide measured timings and accuracy tradeoffs rather than speculative speedups.

2. Are we extracting the right FFT peaks?

Review the full processing chain in `analysis_core.py`: windowing, mean removal, power/log transforms, background estimation, smoothing, thresholding, connected components, peak localization, separation and opposite-spot grouping.

An important concern is **crop-size dependence**. Image **1156** appears to contain one consistent, clearly periodic crystal across many crop sizes, yet detection and classification can change substantially. Smoothing appears influential. See `docs/crop-size-diagnostic.md` and any available diagnostic exports.

Investigate:

- Whether smoothing specified in FFT bins behaves appropriately as reciprocal sampling changes.
- Whether background normalization suppresses genuine weak reflections or promotes noise.
- Whether adjacent reflections merge, broad peaks split, or centroids shift.

- Whether minimum separation and matching tolerances reflect actual peak widths and positional uncertainty.
- Whether the initial detector cutoff excludes useful peaks that auto minimum-strength optimization can never recover.
- Whether stronger peaks dominate threshold selection excessively.

Evaluate alternatives where justified: multiscale detection, local noise/prominence estimates, background-subtracted integrated intensity, watershed separation, subpixel peak fitting, and uncertainty-aware matching. Explain which are appropriate for TEM FFTs and which belong only in direct diffraction.

Keep the processed signal inspectable and preserve provenance back to the original pixels.

3. Auto objectives and weaker second patterns

Rank-auto can raise the threshold until weaker BCC-like evidence disappears, leaving a stronger HCP interpretation. Geometry-auto is intended to retain evidence for one or two patterns, but remains heuristic.

Please examine:

- Whether selecting peaks by the eventual crystal score introduces selection bias.
- Whether global peak-coverage denominators unfairly penalize a valid weaker pattern for failing to explain another pattern.
- How to distinguish two genuinely supported patterns from two labels explaining the same spots.
- Whether two orientations of the **same phase** can actually survive candidate retention and classification.
- Whether primary/secondary geometry searches miss patterns, duplicate one lattice, or favour harmonic/sub-lattice explanations.
- Whether lowering thresholds adds independent structured evidence or merely chance matches.

Simply summing the top two crystal ranks is not an adequate solution unless shared evidence and added model flexibility are handled explicitly.

4. Scoring and model flexibility

Review `phase_comparison.py`, `scale_fitting.py`, `crystal_templates.py` and `peak_evidence.py`.

Pay particular attention to:

- Extra flexibility of HCP and orthorhombic fits versus cubic fits.
- Unidentifiable lattice parameters from a single projection.
- Candidate truncation and the finite zone-axis library.
- Missing-reflection penalties when spots may be obscured, weak or physically absent.
- Weak-peak bonuses, matching radii and potential double counting.
- Shared HCP/BCC explanations that resemble a two-phase result.

Distinguish demonstrable implementation defects from scientific assumptions needing validation.

Requested deliverable

Return:

1. Prioritized findings with file/function references and supporting evidence.
2. A performance breakdown for manual, geometry-auto and rank-auto.
3. A small validation matrix covering clear single crystals, weak second patterns, same-phase orientations, noise and crop-size variation.
4. Recommended immediate fixes versus experimental alternatives.
5. Expected benefits, risks and a minimal implementation sequence.

Use synthetic patterns with known answers alongside real examples. Treat existing phase labels as hypotheses, not ground truth. If data are unavailable, state the limitation rather than inventing benchmark or accuracy results.